

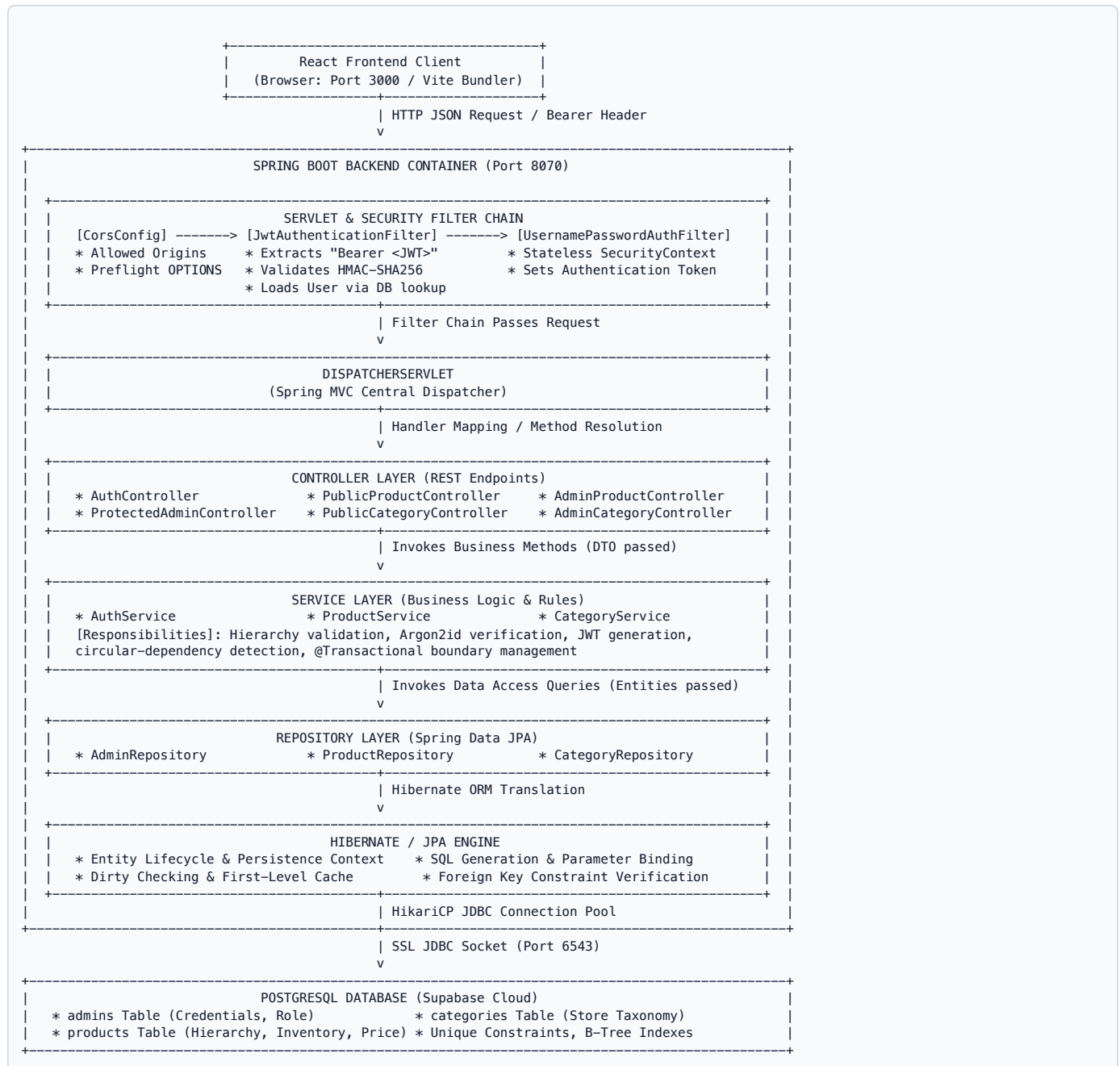
DronesZ Backend — Complete System Walkthrough

Full Architectural, Security, API & Database Reference Guide

Target Stack: Java 8 / Spring Boot 2.7.18 Security: Spring Security & JWT Database: PostgreSQL / Supabase

1. High-Level System Architecture

The DronesZ system is built as a layered, decoupled web application. The frontend communicates with the backend exclusively via standard HTTP REST APIs with JSON payloads and Bearer token headers.



The Layers in Simple Terms

- BEGINNER Frontend:** React single-page app that sends JSON payloads and JWT headers.
- BEGINNER Controller:** Ingests HTTP requests, triggers validation (`@Valid`), and returns HTTP status codes.
- INTERMEDIATE Service:** Enforces business domain logic (e.g. 3-tier product hierarchy, Argon2id password checks).
- INTERMEDIATE Repository:** Spring Data JPA interface generating SQL queries dynamically.

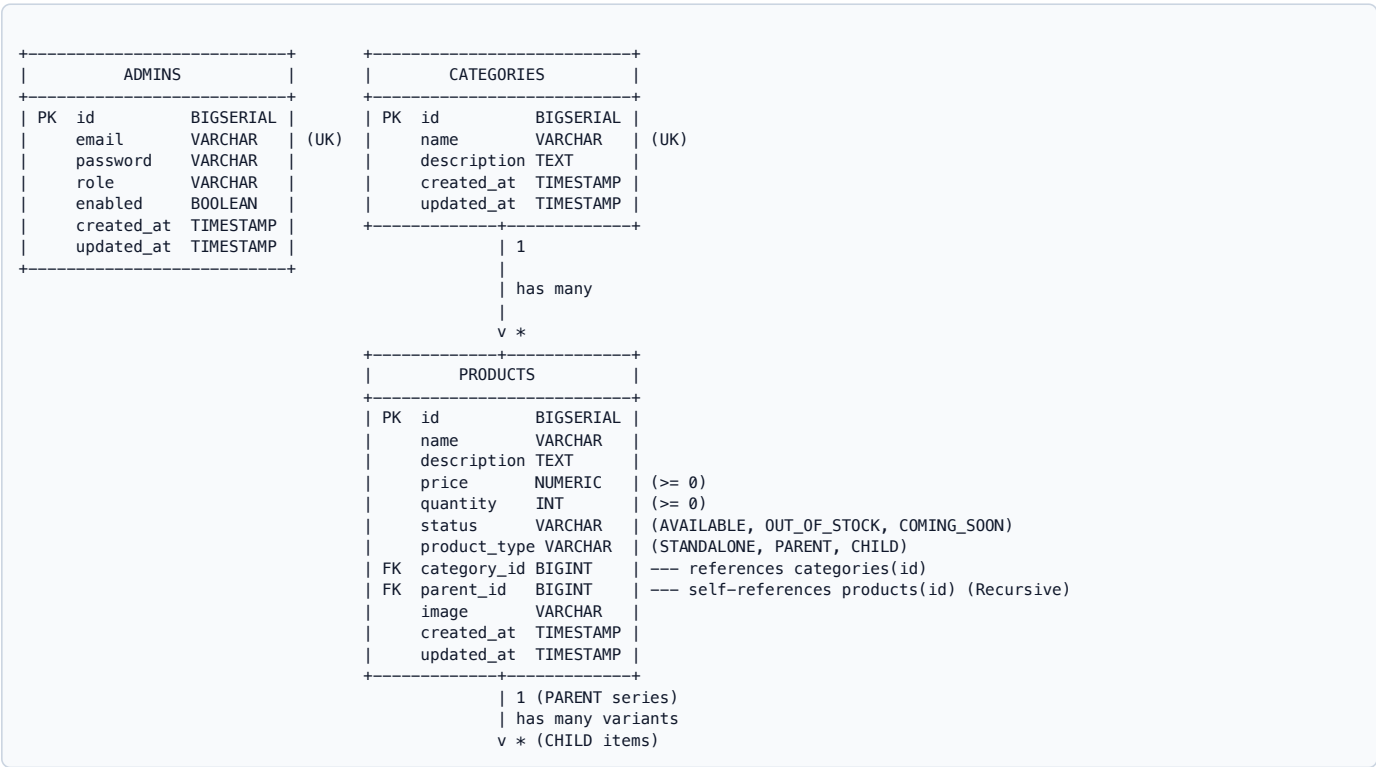
- **ADVANCED** **Hibernate & JPA:** Manages object-relational mapping, dirty checking, first-level caching, and connection transactions.

2. Backend Project & Package Structure

The backend source tree is organized into cohesive functional packages:

Package / Class	Layer	Responsibility	Called By	Calls
<code>config.CorsConfig</code>	Config	Configures allowed cross-origin HTTP origins, headers, and methods.	Spring Security	Security Filter Chain
<code>security.SecurityConfig</code>	Security	Defines URL authorization rules, stateless sessions, Argon2id encoder.	Spring Boot	<code>JwtAuthenticationFilter</code>
<code>security.JwtTokenProvider</code>	Security	Generates, signs, parses, and cryptographically validates JWT tokens.	<code>AuthService</code> , Filter	<code>io.jsonwebtoken</code>
<code>security.JwtAuthenticationFilter</code>	Security	Extracts Bearer token from incoming request and sets <code>SecurityContext</code> .	Servlet Filter Chain	<code>JwtTokenProvider</code> , <code>CustomUserDetailsService</code>
<code>controller.AuthController</code>	Controller	Handles <code>POST /api/auth/admin/login</code> requests.	HTTP Clients	<code>AuthService</code>
<code>controller.AdminProductController</code>	Controller	Handles protected product CRUD operations (<code>/api/admin/products/**</code>).	Admin UI (JWT)	<code>ProductService</code>
<code>controller.PublicProductController</code>	Controller	Handles unauthenticated public catalog queries (<code>/api/products/**</code>).	Public Storefront	<code>ProductService</code>
<code>service.ProductService</code>	Service	Validates 3-tier hierarchy, circular references, and manages transactions.	Controllers	<code>ProductRepository</code> , <code>CategoryRepository</code>
<code>exception.GlobalExceptionHandler</code>	Exception	Catches unhandled runtime exceptions and returns formatted JSON errors.	Spring MVC	<code>ErrorResponse</code>

3. Database Design & Entity Relationships



Key Database Attributes

- admins:** Stores administrative credentials. Unique index on `email` for fast login queries. Password is stored as a secure one-way `Argon2id` hash digest.
- categories:** Stores product taxonomy. Unique constraint on `name` prevents duplicate store sections.
- products:** Features a self-referencing foreign key `parent_id` to support variant model hierarchies. Check constraints enforce non-negative prices and quantities.

4. Authentication & JWT Lifecycle

The Admin Login Flow

1. The admin submits credentials to `POST /api/auth/admin/login`.
2. `AuthController` deserializes the request to a `LoginRequest` DTO and executes `@Valid` checks.
3. `AuthService` fetches the admin record from `AdminRepository` by email.
4. `passwordEncoder.matches()` verifies the raw password against the Argon2id hash.
5. `JwtTokenProvider` generates a cryptographically signed HMAC-SHA256 JWT with subject=email and role= `ADMIN`.
6. The client receives the token and stores it in browser memory / `localStorage`.

Inside the DronesZ JWT Token

Header: `{"alg": "HS256", "typ": "JWT"}`

Payload (Claims):

```
{
  "sub": "admin@example.com",
  "role": "ADMIN",
  "iat": 1724998400,
  "exp": 1725084800
}
```

Signature: `HMACSHA256(base64(header) + "." + base64(payload), secretKey)`

How the JWT Filter Intercepts Requests

`JwtAuthenticationFilter` extends `OncePerRequestFilter`:

```
String jwt = getJwtFromRequest(request);
if (StringUtils.hasText(jwt) && tokenProvider.validateToken(jwt)) {
    String email = tokenProvider.getEmailFromToken(jwt);
    UserDetails userDetails = userDetailsService.loadUserByUsername(email);
    if (userDetails.isEnabled()) {
        UsernamePasswordAuthenticationToken authentication =
            new UsernamePasswordAuthenticationToken(userDetails, null, userDetails.getAuthorities());
        SecurityContextHolder.getContext().setAuthentication(authentication);
    }
}
filterChain.doFilter(request, response);
```

5. The 3-Tier Product Hierarchy System

DronesZ enforces a strict 3-tier catalog hierarchy:

Product Type	Parent ID Allowed?	Displayed on Main Store?	Purpose & Description
STANDALONE	No (Must be null)	Yes	Independent item with its own price, stock, and purchasing button (e.g. Drone Frame).
PARENT	No (Must be null)	Yes	Series header grouping variants (e.g. <i>X-Rotor Motor Series</i>). Has no direct price.
CHILD	Yes (Must point to PARENT)	No (Hidden from main catalog)	Specific model variant (e.g. <i>1850KV</i> vs <i>2400KV</i>). Shown only inside parent series view.

6. Master REST API Inventory

Method	Endpoint	Auth Level	Role	Controller	Purpose
GET	/	Public	None	HomeController	Returns API status and Swagger documentation URLs.
POST	/api/auth/admin/login	Public	None	AuthController	Admin login endpoint issuing signed JWT tokens.
GET	/api/products	Public	None	PublicProductController	Lists STANDALONE and PARENT products for public catalog.
GET	/api/products/{id}	Public	None	PublicProductController	Retrieves single product details by primary key ID.
GET	/api/products/{id}/children	Public	None	PublicProductController	Fetches all child variants belonging to a parent series.
GET	/api/categories	Public	None	PublicCategoryController	Returns list of all available product categories.
GET	/api/admin/me	Protected	ROLE_ADMIN	ProtectedAdminController	Returns authenticated administrator profile.
GET	/api/admin/products	Protected	ROLE_ADMIN	AdminProductController	Lists all products with pagination, search, and type filters.
POST	/api/admin/products	Protected	ROLE_ADMIN	AdminProductController	Creates a new product with hierarchy validation.
PUT	/api/admin/products/{id}	Protected	ROLE_ADMIN	AdminProductController	Updates product data & prevents circular hierarchy.
DELETE	/api/admin/products/{id}	Protected	ROLE_ADMIN	AdminProductController	Deletes product (Blocked if child variants exist).
GET	/api/admin/categories	Protected	ROLE_ADMIN	AdminCategoryController	Lists all categories for administration dashboard.
POST	/api/admin/categories	Protected	ROLE_ADMIN	AdminCategoryController	Creates new category (Unique name enforced).
DELETE	/api/admin/categories/{id}	Protected	ROLE_ADMIN	AdminCategoryController	Deletes category (Blocked if products assigned).

7. Security Audit & Threat Model

Security Area	Current Code Status	Risk Level	Explanation & Remediation
Password Hashing	Argon2id (BouncyCastle)	SECURE	Memory-hard hashing prevents GPU/ASIC brute-force cracking.
Token Storage	localStorage	HIGH RISK	Vulnerable to token theft via XSS. Move to HttpOnly cookies.
Rate Limiting	None	HIGH RISK	Login endpoint is vulnerable to automated brute-force attacks.
Key Padding	Zero-pads short keys	MEDIUM RISK	Weak keys under 32 bytes are padded. Require >= 256 bits on startup.
SQL Injection	JPA Specifications	SECURE	All queries use parameterized statements. Zero string concatenation.

8. "DronesZ Backend in One Page" (Summary Sheet)

- **Core Architecture:** 3-tier Spring Boot 2.7.18 REST API connected to a cloud PostgreSQL database (Supabase) and consumed by a React TypeScript single-page application.
- **Authentication Flow:** Admins authenticate via `POST /api/auth/admin/login`. The backend verifies credentials using `Argon2id` password hashing and issues a 24-hour HMAC-SHA256 signed JWT Bearer token.
- **Security Interception:** `JwtAuthenticationFilter` extends `OncePerRequestFilter`. It parses the incoming `Authorization: Bearer <token>` header, validates the signature, loads the admin entity, and sets Spring Security's `SecurityContextHolder`.
- **Authorization Model:** Role-Based Access Control (RBAC). `/api/admin/**` endpoints require `ROLE_ADMIN` authority. Missing tokens return `401 Unauthorized`; insufficient roles return `403 Forbidden`.
- **3-Tier Product Hierarchy:**
 - `STANDALONE`: Individual product with price, stock, and purchase flow.
 - `PARENT`: Variant category header (e.g. *Motor Series*) with no direct price.
 - `CHILD`: Specific model variant (e.g. *1850KV*) hidden from main catalog view and accessible exclusively via `/api/products/{parentId}/children`.
- **Request Pipeline:** `React UI` → `CORS / Security Filter` → `Controller` → `DTO Validation` → `Service (@Transactional)` → `Repository (JPA Criteria)` → `Hibernate ORM` → `PostgreSQL`.